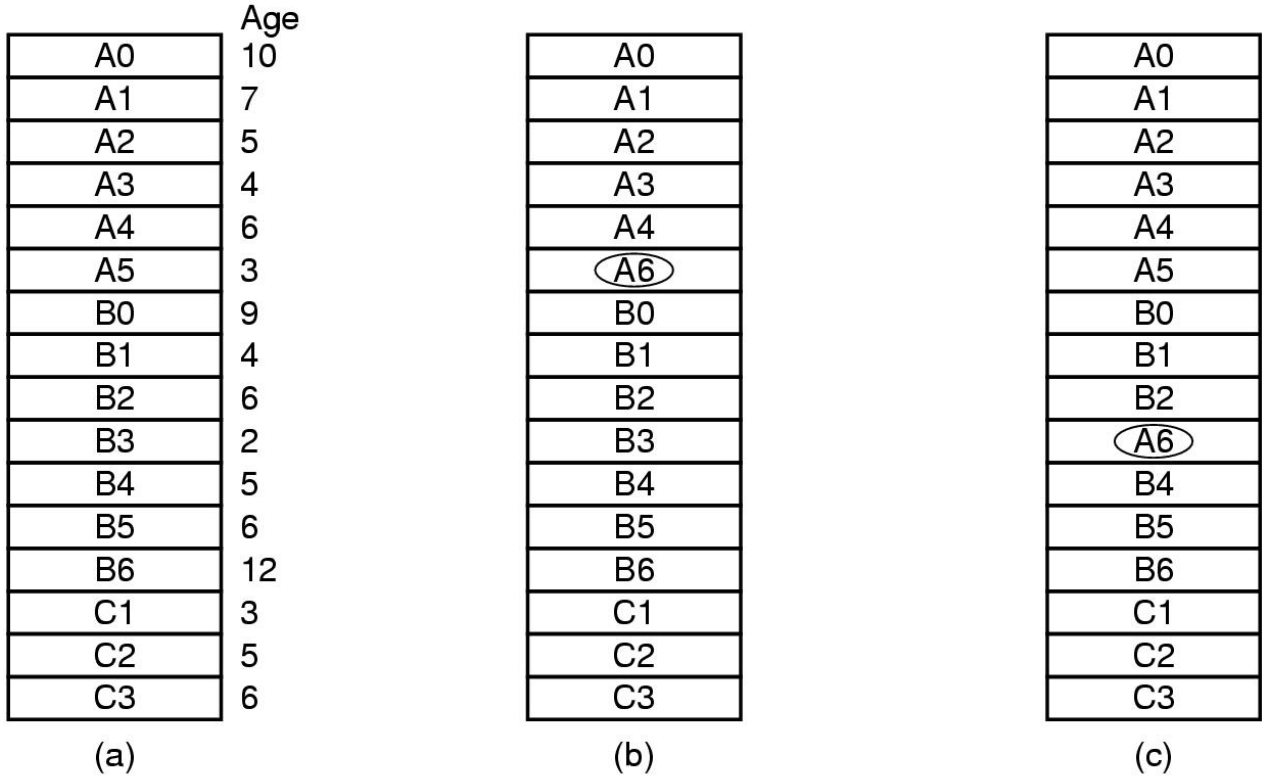
Chapter 3

Memory Management

- Fetch Strategies
- Page Replacement
  - -Optimal -FIFO -LRU -NRU -Second-chance
- Design Issues for paging systems
- Implementation Issues
- Segmentation
- Summary

Design Issues for Paging Systems

Local versus Global Allocation Policies (1)



a) Original configuration
b) Local page replacement
c) Global page replacement

Load Control

- Despite good designs, system may still thrash
- Some processes need more memory
  but no processes need less
- Solution :
  Reduce a number of processes competing for memory
  - swap one or more to disk, divide up pages they held
  - reconsider degree of multiprogramming

Page Size (1)

Small page size

- Advantages
  - less internal fragmentation
  - less unused program in memory
- Disadvantages
  - programs need many pages, larger page tables

Page Size (2)

- Overhead due to page table and internal fragmentation

$$overhead = \frac{s \cdot e}{p} + \frac{p}{2}$$

page table space

internal fragmentation

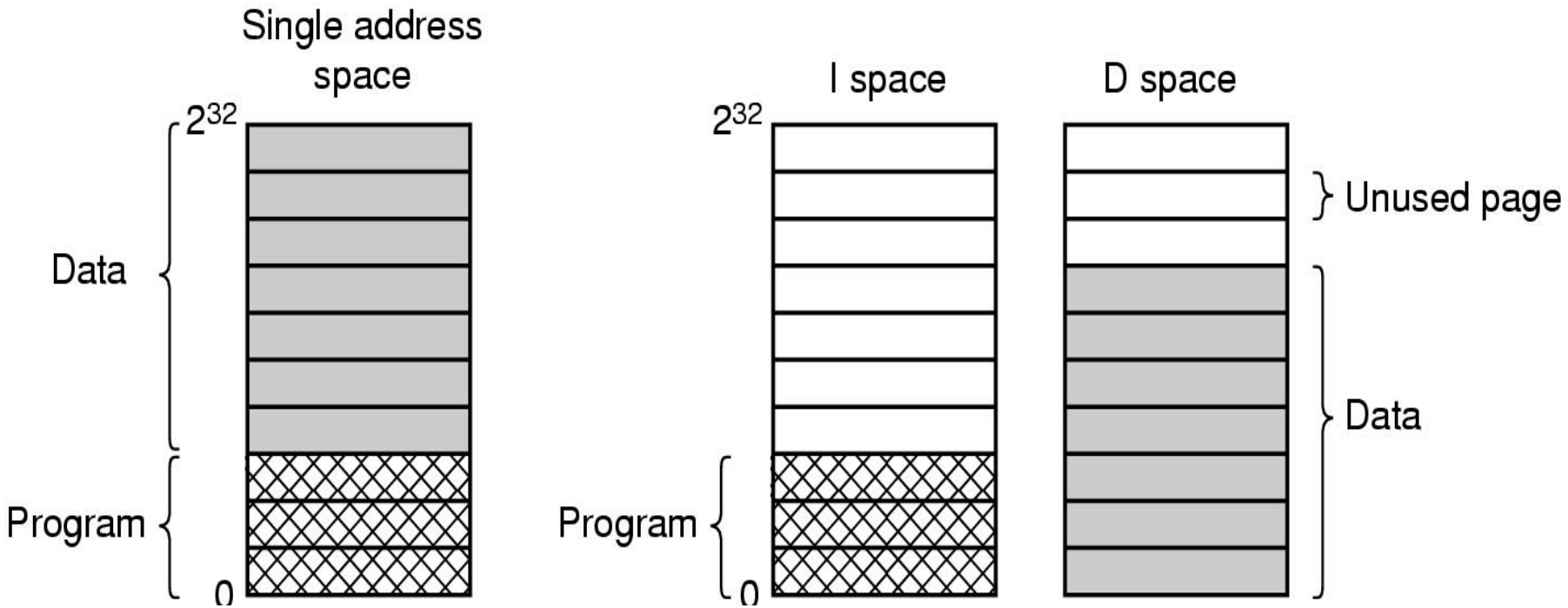
- Where
  - *s* = average process size in bytes
  - *p* = page size in bytes
  - *e* = page entry size in bytes

Optimized when

$$p = \sqrt{2se}$$

S=1MB, e=8, p=4KB

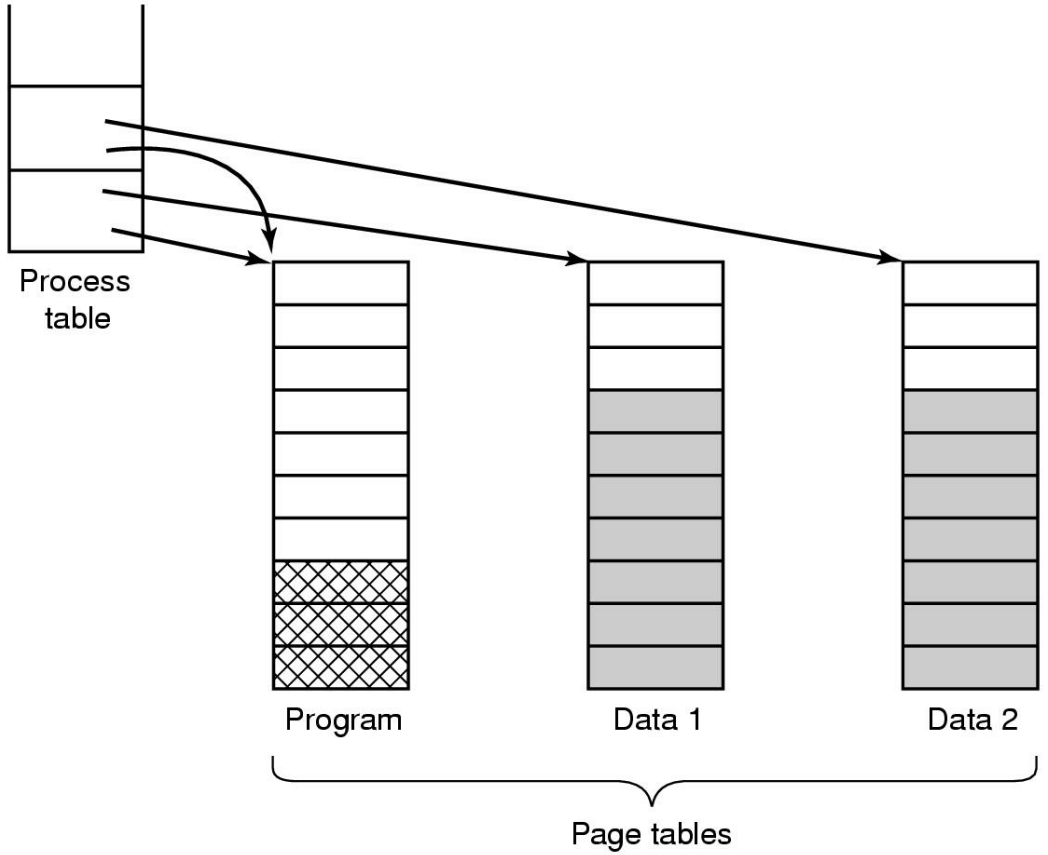
Separate Instruction and Data Spaces



- One address space
- Separate I and D spaces

Shared Pages

Separate instruction and data space, easy to share



Two processes sharing same program sharing its page table

Cleaning Policy

- Need for a background process, paging daemon
  - periodically inspects state of memory
- When too few frames are free
  - selects pages to evict using a replacement algorithm
- It can use same circular list (clock)
  - as regular page replacement algorithm but with diff ptr (front hand & back hand)

Implementation Issues

Operating System Involvement with Paging

Four times when OS involved with paging

1. Process creation
   - determine program size
   - create page table
2. Process execution
   - MMU reset for new process
   - TLB flushed
3. Page fault time
   - determine virtual address causing fault
   - swap target page out, needed page in
4. Process termination time
   - release page table, pages

Page Fault Handling (1)

1. Hardware traps to kernel, saving the PC on the stack
2. General registers saved
3. OS determines which virtual page needed
4. OS checks validity of address, seeks page frame
5. If selected frame is dirty, write it to disk, suspend the process

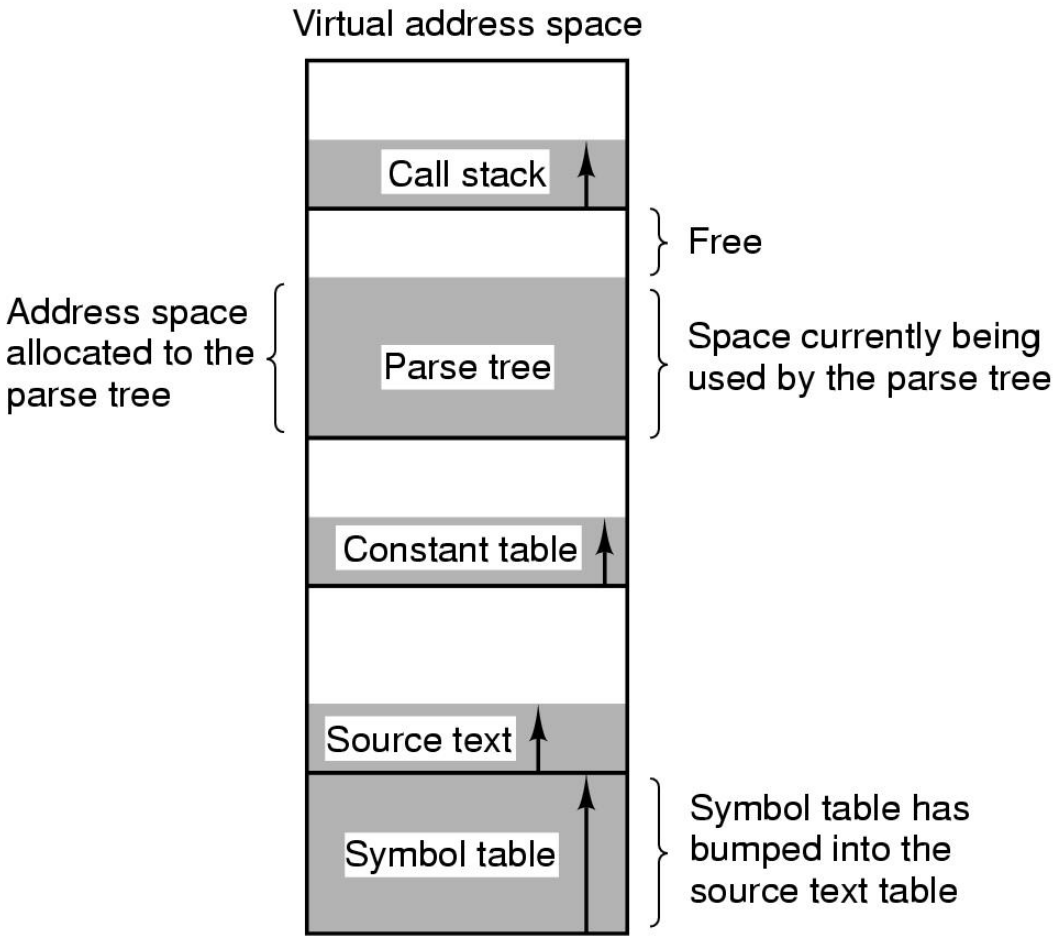
Page Fault Handling (2)

6. OS brings schedules new page in from disk. While page loading, process still suspend
7. Disk interrupts, page tables updated
8. Faulting instruction backed up to when it began
9. Faulting process scheduled
10. Registers restored
11. Program continues

Segmentation

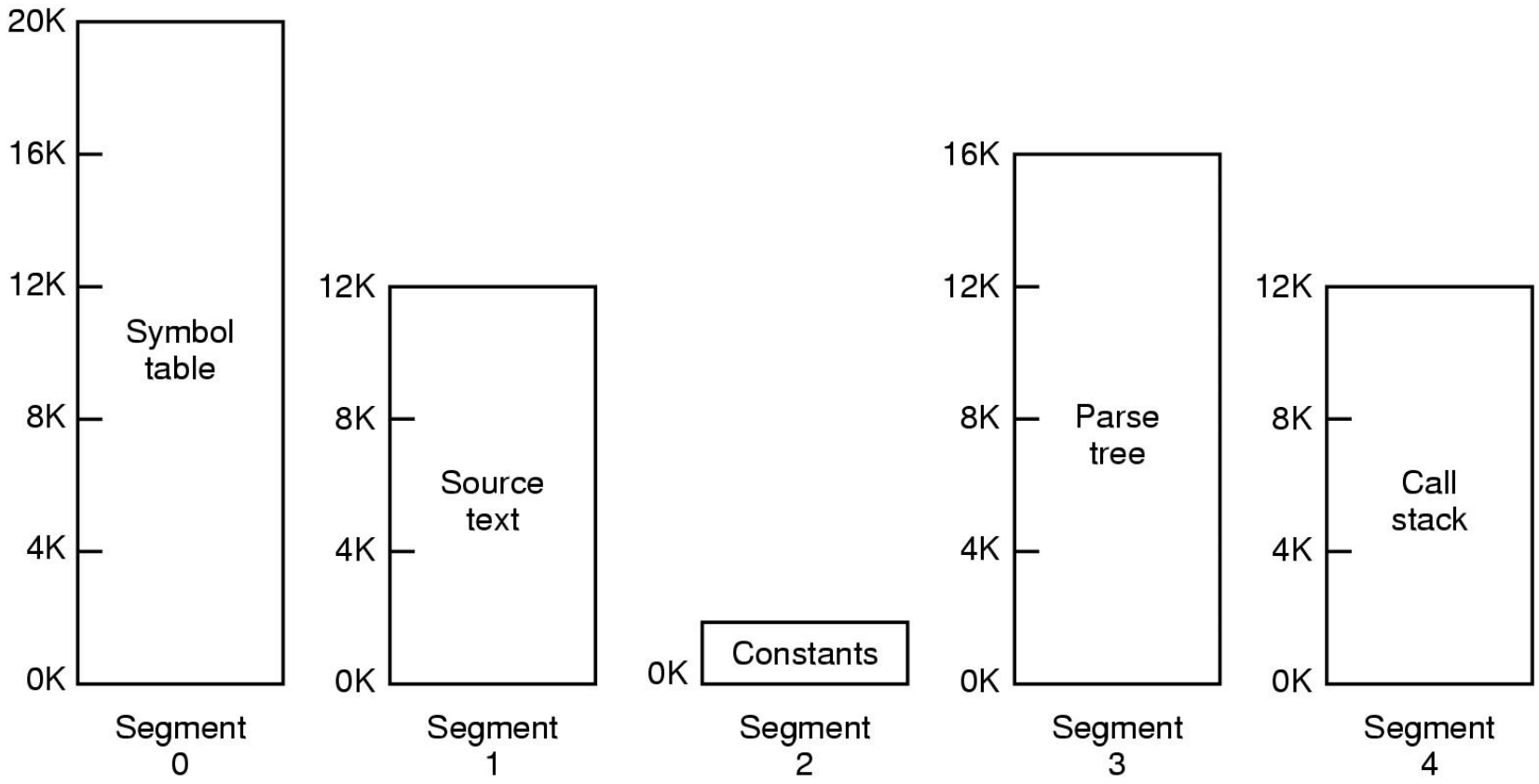
- The virtual memory solution so far is paging (one-dimensional)
- For some applications, having two or more virtual address spaces may be much better than having only one.
  - For example, a compiler has many tables that are built up as compilation proceeds, such as source text, symbol table, constant table, parse tree and stack.

Segmentation (1)



- One-dimensional address space with growing tables
- One table may bump into another

Segmentation (2)

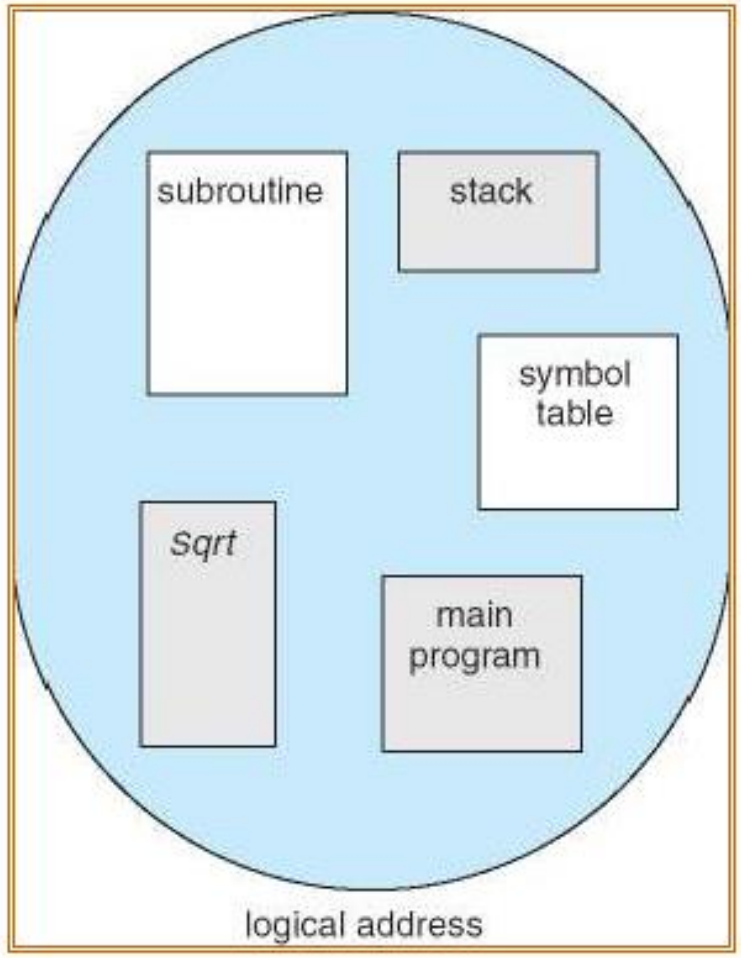


Allows each table to grow or shrink, independently

Segmentation (3)

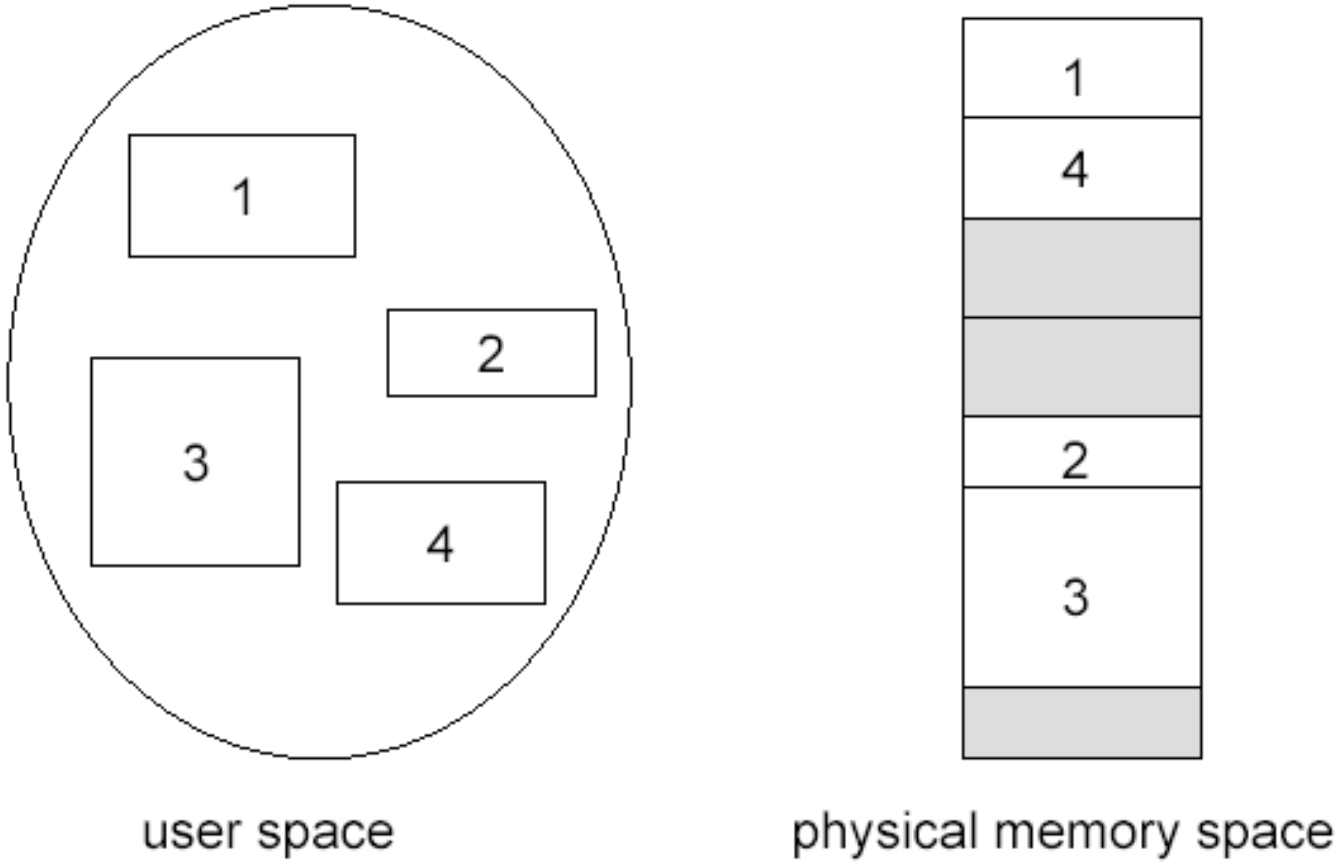
- Segmentation is a memory-management scheme that supports user view of memory.
- Pages are fixed in size, segments are variable sized.
- A program is a collection of segments. A segment is a logical unit such as:
  - Main program
  - Procedure
  - Function
  - Symbol table
  - Stack

Segmentation (4)



User's view of a program

Segmentation (5)



Logical view of segment

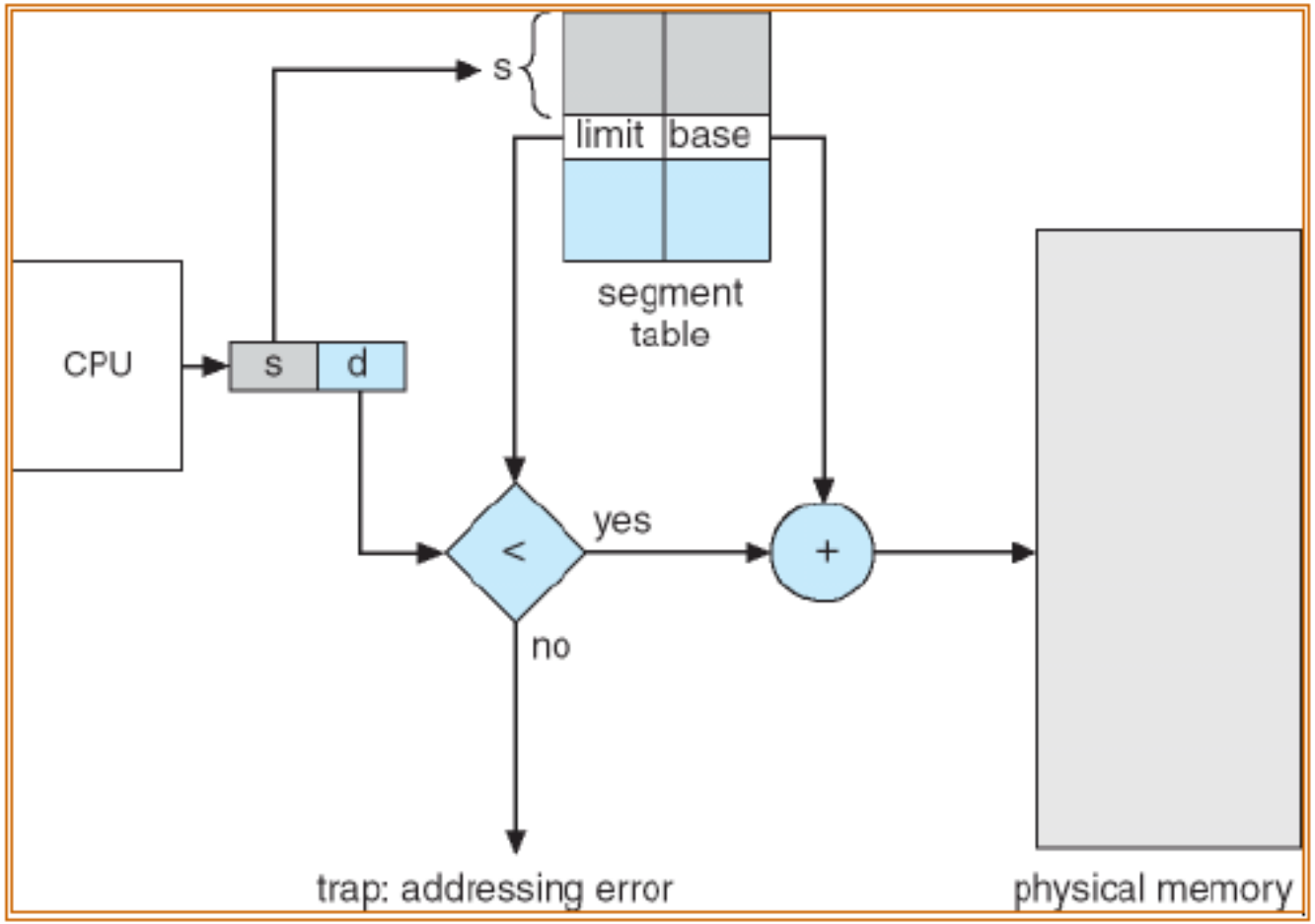
Segmentation Architecture (1)

- Logical address consists of two parts:

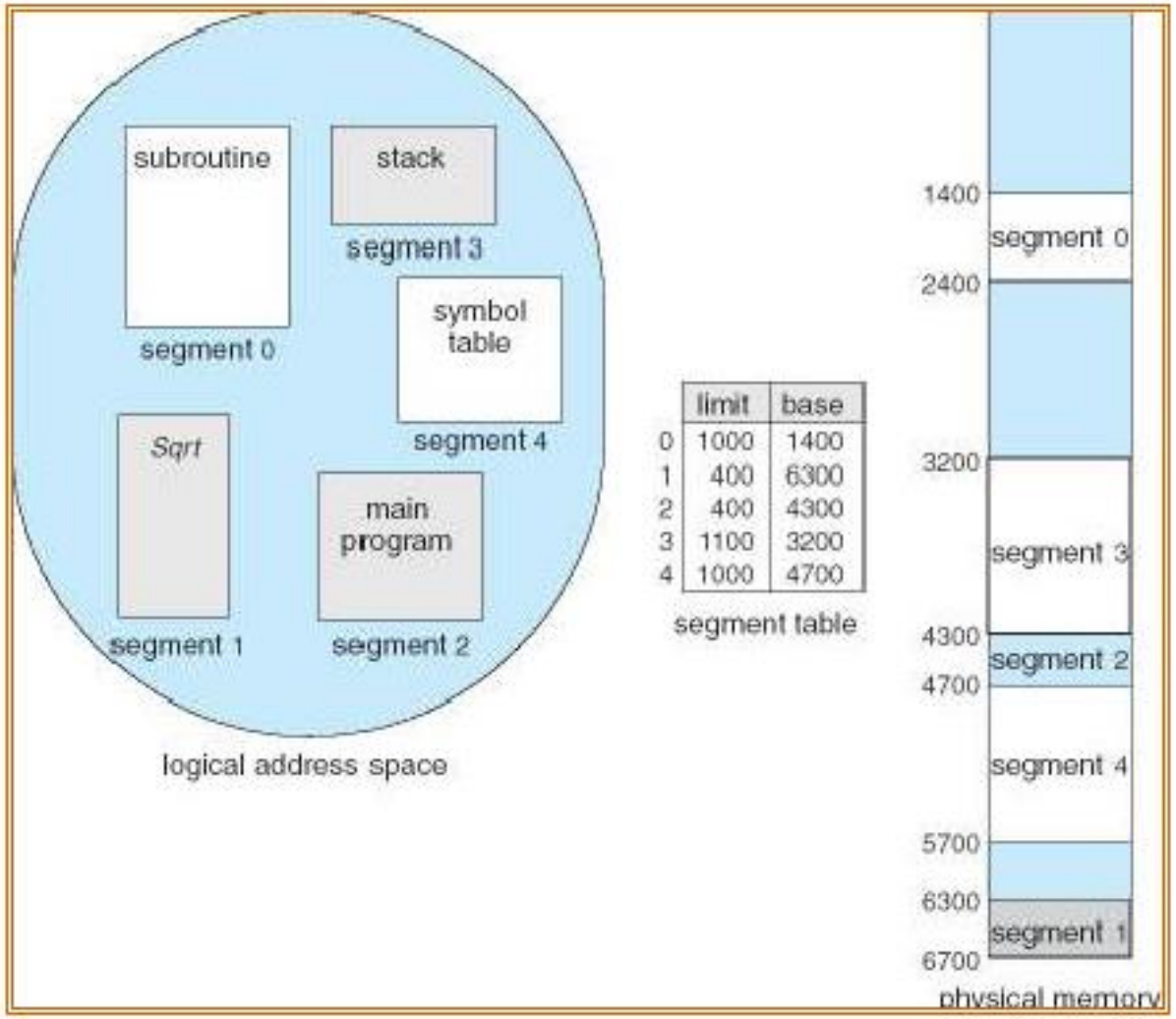
  <segment-number, offset>

- **Segment table**: maps two-dimensional user-defined addresses into one-dimensional physical addresses; each table entry has:
  - **Base**: contains the starting physical address where the segments reside in memory
  - **Limit**: specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory

Address Translation



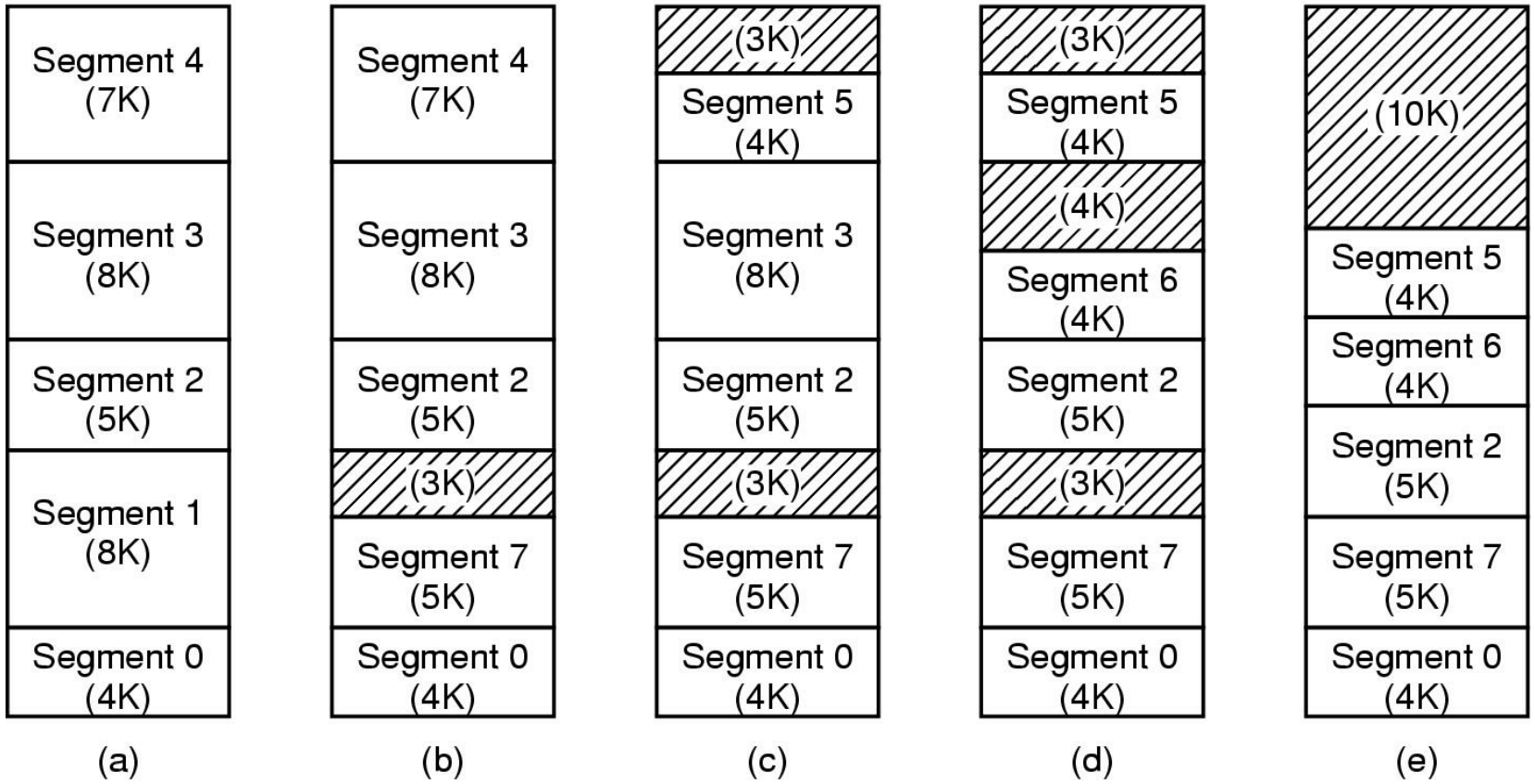
Example of Segmentation



Segmentation Architecture (2)

- Protection
  - With each entry in segment table associate
    - validation bit = 0 ⇒ illegal segment
    - read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem

Implementation of Pure Segmentation



(a)-(d) Development of checkerboarding

(e) Removal of the checkerboarding by compaction

Paging vs. Segmentation

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

Comparison of paging and segmentation

Advantage of Segmentation

- May be possible to shrink/grow segments
- Each segment can be given its own protection information...this is a much easier protection scheme than trying to protect each page of memory
- Linking programs is a trivial task
- Code can be shared between processes easily. Just load the code segment once. Copies of the same program access the same segment.

Disadvantage of Segmentation

- Programmer must be aware of the memory model in use (at the assembly level, anyway)
- Just like with swapping systems, fragmentation can waste much memory.
- Segments may be too large to fit in physical memory

Segmentation with paging (1)

- Segmentation in virtual memory, paging in real memory
- A segment is composed of pages
- An address has three components

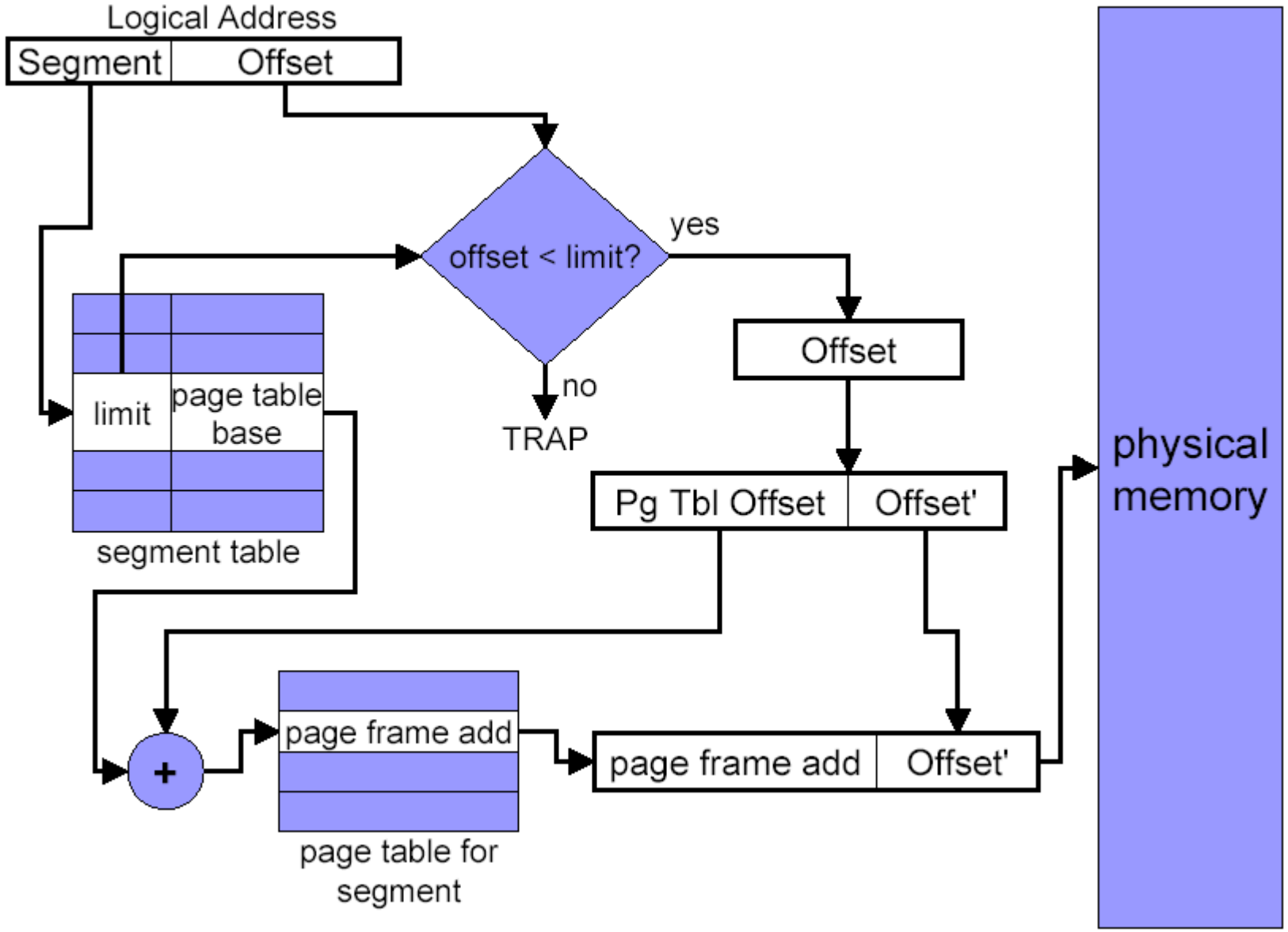


- The real memory contains only the demanded pages of a segment, not the full segment

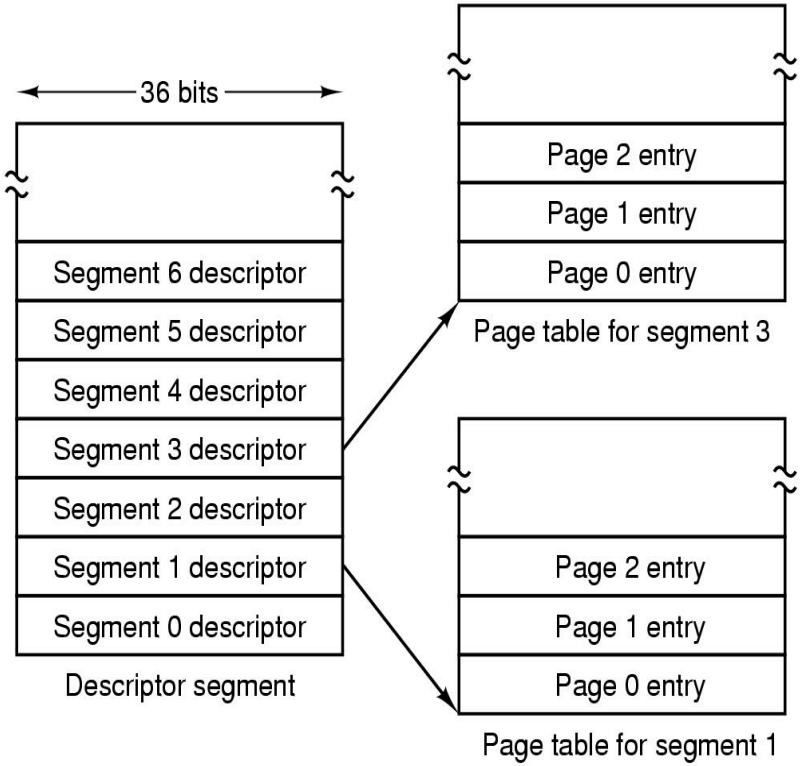
Segmentation with paging (2)

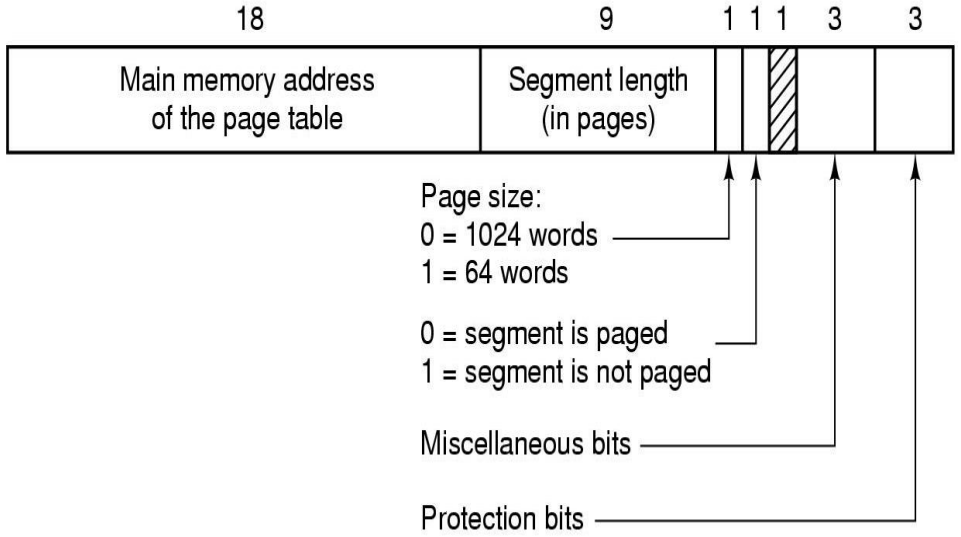
- Required
  - Each process needs a segment table
    - This table may be segmented and paged itself!
  - Each entry in the segment table points to the page table for that segment
    - Like before, this may be a multi-level page table

Address Translation



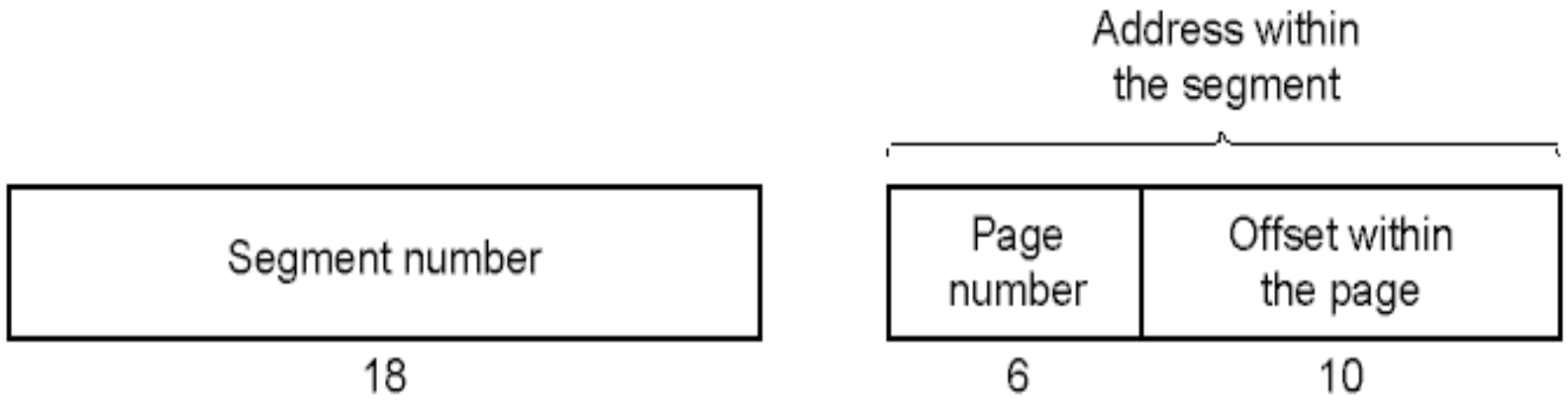
Segmentation with Paging: MULTICS





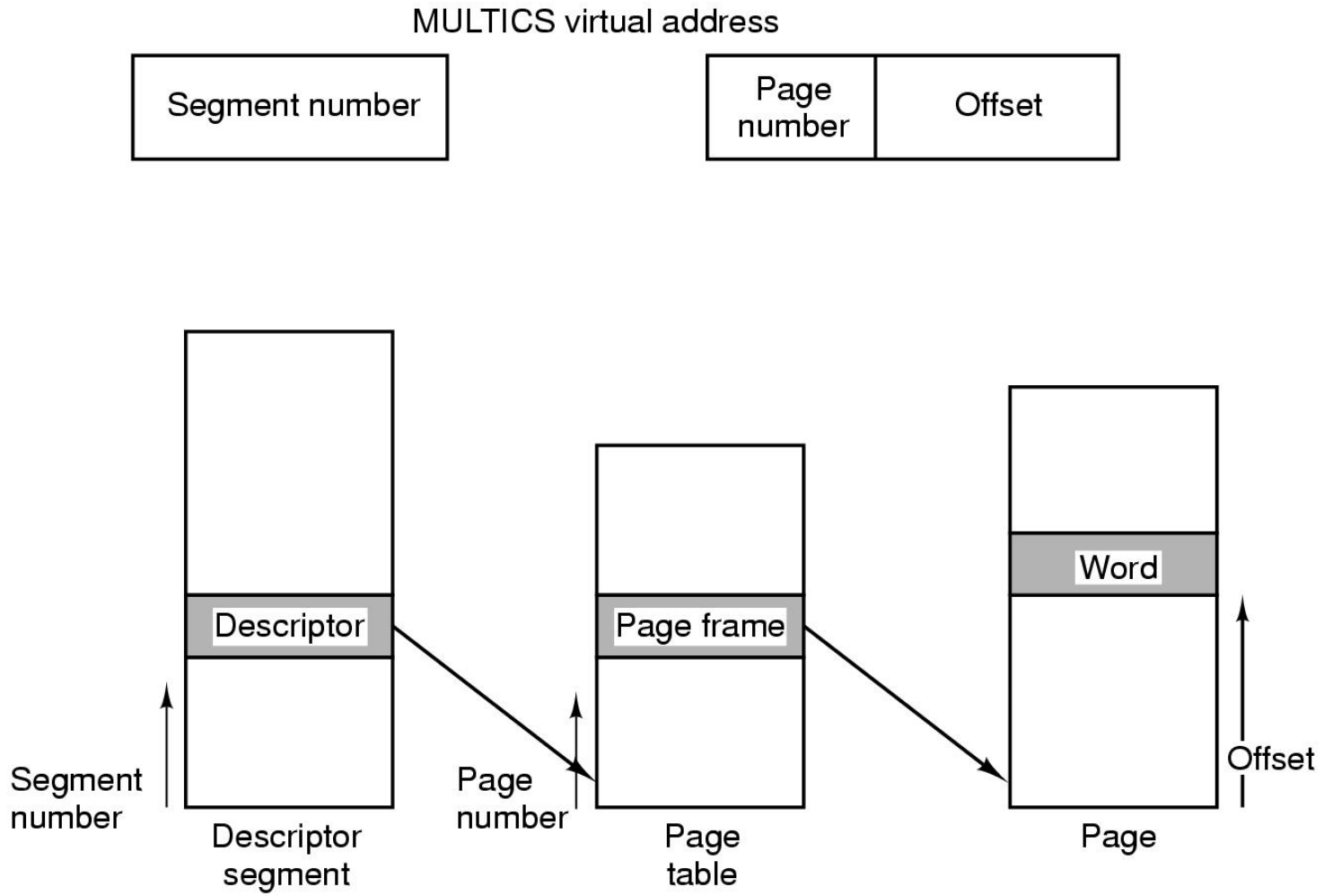
- Descriptor segment points to page tables
- Segment descriptor – numbers are field lengths

Segmentation with Paging: MULTICS



A 34-bit MULTICS virtual address

Segmentation with Paging: MULTICS



Conversion of a 2-part MULTICS address into a main memory address

Segmentation with Paging: MULTICS

Comparison field: Segment number	Comparison field: Virtual page	Page frame	Protection	Age	Is this entry used?
4	1	7	Read/write	13	1
6	0	2	Read only	10	1
12	3	1	Read/write	2	1
					0
2	1	0	Execute only	7	1
2	2	12	Execute only	9	1

- Simplified version of the MULTICS TLB
- Existence of 2 page sizes makes actual TLB more complicated

Summary

- Fetch Strategies
- Page Replacement
  - Optimal
  - FIFO, Second-chance, Clock
  - NRU, LRU, NFU, Aging
  - Working Set, WSClock
- Design Issues for paging systems
- Implementation Issues
- Segmentation